GUIA TÉCNICO

Transformers, Atenção e BERT

Propósito — Conteúdo didático autoral — notas técnicas e exemplos independentes. Material de apoio histórico não é reproduzido neste PDF.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. Fundamentos e objetivo
2. Modelo e escolhas metodológicas
3. Receitas Python reproduzíveis
4. Interpretação e limitações
5. Exercícios de aplicação
6. Checklist de qualidade

01 | O problema e seu contexto

Transformers, Atenção e BERT é estudado como técnica computacional, com hipóteses explícitas, dados de demonstração e análise crítica dos resultados.

Cuidados metodológicos

O cálculo local da atenção ilustra pesos e combinações, mas não constitui execução de BERT. Atenção, isoladamente, não prova explicabilidade.

02 | Fundamentos e modelo

Objetivos de aprendizagem

- Entender atenção, tokens e embeddings posicionais.
- Calcular atenção escalada num exemplo NumPy.
- Compreender BERT como codificador contextual e limites do exemplo.

Fundamentos conceituais

Transformers substituem recorrência estrita por operações de atenção e projeções, permitindo combinar informações de múltiplas posições. Em self-attention, consultas (Q), chaves (K) e valores (V) são transformações aprendidas de representações de tokens. BERT é um modelo de codificação bidirecional pré-treinado; seu treinamento e fine-tuning são diferentes de executar uma demonstração numérica de atenção.

Modelo matemático e interpretação

$\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d_k})V$. O fator $\sqrt{d_k}$ controla a escala do produto escalar. Em cenários autoregressivos, uma máscara causal bloqueia posições futuras.

Estudo guiado

Pergunta de investigação: como demonstrar o conceito central com um exemplo pequeno, auditável e reproduzível?

Nas células seguintes, inspecione as entradas, execute os procedimentos e explique com suas próprias palavras o resultado. Ao trocar o texto ou o parâmetro, observe se o comportamento permanece coerente. Esses resultados são ilustrações pedagógicas, não benchmarks.

Implementação reproduzível

O código abaixo usa textos sintéticos ou bibliotecas locais. Não executa downloads nem acessa APIs externas. Exemplos opcionais só rodam quando a dependência já está disponível; a ausência deles não interrompe o notebook.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Exercícios orientados

1. Compreensão: explique qual informação o método preserva e qual descarta. Relacione a resposta com os objetivos do capítulo.
2. Experimento: altere uma frase e um parâmetro do código. Registre a diferença e uma hipótese explicativa.
3. Avaliação crítica: descreva uma situação em que a técnica geraria uma resposta inadequada. Proponha uma mitigação.
4. Transferência: conecte o conceito a uma disciplina anterior e identifique o que não precisa ser revisto.

Resolução comentada (diretriz de resposta): o método representa somente um recorte do fenômeno linguístico; mudanças de entrada alteram a saída segundo a regra estudada. Uma boa análise identifica perda de informação, contexto e risco de generalização. Para a resposta específica, justifique com os valores efetivamente obtidos na execução acima.

Erros comuns, limitações e boas práticas

A demonstração NumPy não é um Transformer completo. A instalação de Transformers e pesos pré-treinados é opcional e pode exigir armazenamento, memória e conexão.

- Separe demonstração de produção: datasets sintéticos não demonstram generalização.
- Registre versões das dependências e parâmetros; não inclua credenciais no notebook.
- Consulte a documentação oficial antes de adaptar códigos históricos de 2021.

Laboratório adicional — Atenção scaled dot-product em NumPy

Experimento independente do estado das células anteriores. Use-o como verificação e explique seus limites; resultados em conjuntos sintéticos não são estimativas de desempenho real.

03 | Cookbook Python

Softmax estável

```
import numpy as np
x = np.array([1.0,2.0,3.0]); z=x-x.max()
a = np.exp(z)/np.exp(z).sum()
print(a, a.sum())
```

Atenção didática

```
import numpy as np
Q=np.array([[1.,0.],[0.,1.]])
K=Q.copy(); V=np.array([[2.,1.],[1.,3.]])
S=Q@K.T/np.sqrt(Q.shape[1]); E=np.exp(S-S.max(axis=1,keepdims=True))
print((E/E.sum(axis=1,keepdims=True))@V)
```

04 | Interpretação e limites

O cálculo local da atenção ilustra pesos e combinações, mas não constitui execução de BERT. Atenção, isoladamente, não prova explicabilidade.

05 | Exercícios com respostas orientadoras

1. Altere uma entrada ou parâmetro e descreva o comportamento observado.

Resposta orientadora: mantenha os demais elementos constantes, documente a mudança e explique qual resultado deveria variar pela hipótese escolhida.

2. Cite uma situação em que a técnica falha ou transmite confiança excessiva.

Resposta orientadora: identifique domínio, idioma, quantidade de dados, formato da entrada, possíveis erros sistemáticos e um teste que exponha o limite.

3. Escreva um teste de sanidade que possa ser executado antes de analisar os resultados.

Resposta orientadora: estabeleça forma, tipo, dimensões ou invariantes esperados para os dados, e uma observação que invalidaria a conclusão.

06 | Referências e consulta

Material construído a partir de exercícios autorais da biblioteca, com o conteúdo histórico utilizado apenas como apoio privado. Documentação técnica: Python (docs.python.org), scikit-learn (scikit-learn.org), NLTK (nltk.org), spaCy (spacy.io).

Portfólio GitHub: https://github.com/eduardojr-pereira/data-science-portfolio

Site pessoal: https://eduardo-pereira.webflow.io/